



UNIVERSITÉ SIDI MOHAMED BEN ABDELLAH FACULTÉ DES SCIENCES ET TECHNIQUES FES

« Ingénierie Electrique et Systèmes Embarqués »

COMPTE RENDU: TP SOFTWARE EMBARQUÉ

Préparé par :

FRANKLIN HAMUNYEMBA
AMANSOUR MOHAMMED
BENCHEKROUN LAKRIMI MOHAMED

Supervisé par :

Pr. Aicha ALAMI HASSANI

Introduction

Ce travail pratique a pour objectif principal de nous initier à la programmation de systèmes embarqués sous Linux à travers la plateforme FOX G20 d'ACMESystem. Cette carte, basée sur un processeur ARM9 AT91SAM9G20 cadencé à 400 MHz, représente un excellent support pédagogique pour appréhender les concepts fondamentaux du développement embarqué réel : cross-compilation, accès aux périphériques matériels, communication réseau et contrôle en temps réel.

Ce TP nous a permis de passer de la théorie à la pratique en maîtrisant l'ensemble de la chaîne de développement :

- Démarrage et configuration d'un système Linux embarqué
- Accès à la carte via console série (port debug) et réseau (SSH/Ethernet)
- Contrôle des GPIO en ligne de commande (sysfs) et en langage C
- Cross-compilation pour architecture ARM avec arm-none-eabi-gcc
- Mise en œuvre d'un serveur web embarqué
- Réalisation de trois projets concrets : chenillard lumineux, LED pilotée par bouton poussoir, et compteur 0–9 sur afficheur 7 segments

L'utilisation de la carte FOX G20, avec sa richesse en périphériques (Ethernet, USB, GPIO, I²C, SPI, ADC, etc.) et sa consommation réduite (0,4 W), illustre parfaitement les exigences modernes des applications IoT, domotique, automatisme industriel et objets connectés. Ce TP constitue ainsi une transition idéale vers des projets plus complexes impliquant des communications réseau avancées (MQTT, WebSocket) ou des interfaces graphiques embarquées.

Objectifs pédagogiques atteints :

- Maîtriser la communication avec un système embarqué Linux
- Programmer en bas niveau (GPIO) et haut niveau (serveur web)
- Développer, compiler et déployer du code pour architecture ARM
- Réaliser des applications interactives matériel/logiciel

Ce rapport présente les différentes étapes réalisées, les résultats obtenus, ainsi que les difficultés surmontées lors de cette séance riche en apprentissages concrets.

Table des matières

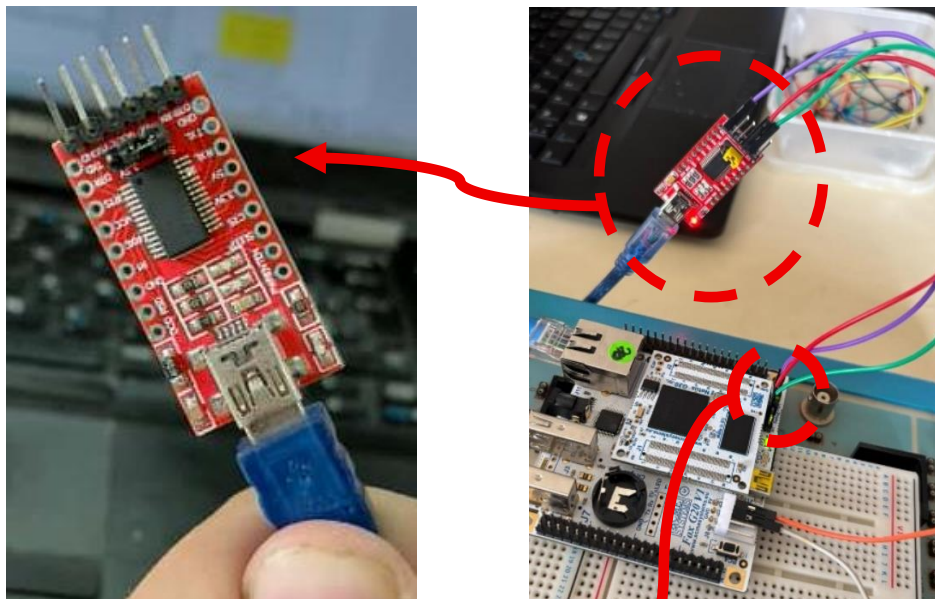
Introduction	2
Initialisation du matériel	5
Connexion série de FOX G20 via FTDI	5
Câblage physique effectué :	5
Connexion série de FOX G20 via Réseaux	6
Installation des applications pré-requises :	7
Connexion au Wifi :	8
Manipulation 1 : LED Clignotante.....	14
But de la manipulation :	14
Logiciels embarqués :	14
Aperçu	14
Fonctions utilisées	14
Fonction principale main()	15
Réalisation physique.....	16
Configuration	17
Conclusion Partielle	18
Manipulation 2 : Chenillard Lumineux.....	19
But de la manipulation :	19
Logiciels embarqués :	19
Aperçu	19
Fonction principale main()	19
Réalisation physique.....	20
Configuration	20
Conclusion Partielle	21
Manipulation 3 : Compteur	22
But de la manipulation :	22
Logiciels embarqués :	22
Aperçu	22

Fonction principale main()	22
Réalisation physique.....	23
Configuration	23
Conclusion Partielle	24
Conclusion Générale	24

Initialisation du matériel

Connexion série de FOX G20 via FTDI

L'accès à la **console de débogage** de la carte FOX G20 est réalisé par le **port UART0**, via un **câble USB-UART FTDI FT232RL**. Cette interface permet une **communication bidirectionnelle** entre le PC hôte et le système Linux embarqué à une vitesse de **115200 bauds**.



Câblage physique effectué :

Broche FTDI	Connexion	Broche FOX G20	Fonction
TXD	→	RXD	Réception données (carte)
RXD	→	TXD	Transmission données (carte)
GND	→	GND	Masse commune

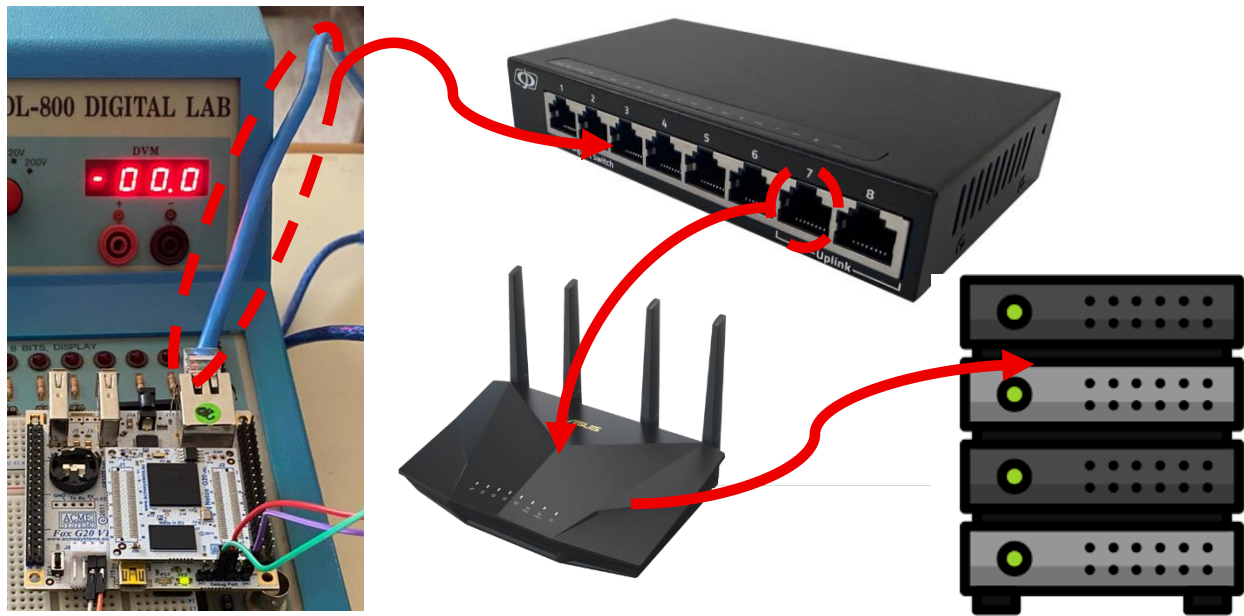
Remarque importante :

Le croisement TX/RX est nécessaire car l'émetteur (TX) d'un appareil doit envoyer les données au récepteur (RX) de l'autre.

Cela permet d'assurer une communication correcte entre les deux dispositifs.

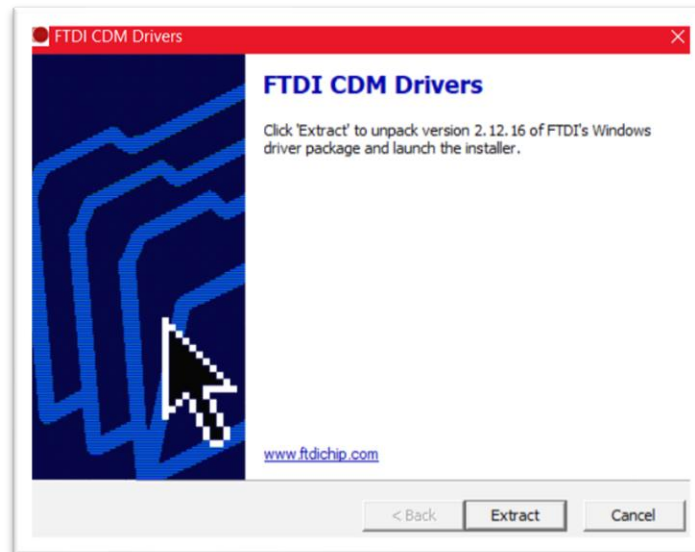
Connexion série de FOX G20 via Réseaux

Afin d'assurer que la carte FOX G20 communique avec le PC, on a la reliée au routeur via un switch intermédiaire au moyen d'une connexion filaire utilisant un câble Ethernet RJ45. Le routeur, configuré en serveur DHCP, attribue automatiquement des adresses IP aux deux appareils : une adresse au PC hôte via la connexion Wi-Fi, et une autre à la carte FOX G20 via la connexion Ethernet. Ce réseau permet également l'accès à un serveur central connecté au routeur, sur lequel sont hébergés et exécutés l'ensemble des fichiers et services associés au projet.

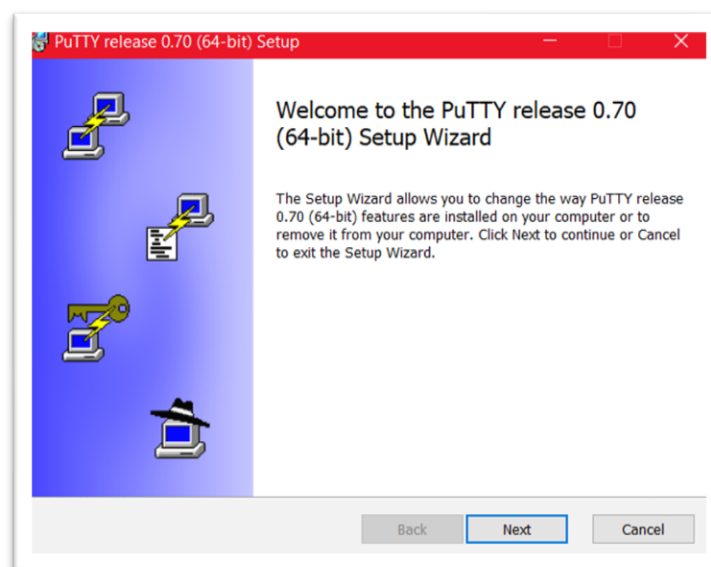


Installation des applications pré-requises :

Tout d'abord, nous avons commencé par installer le pilote CDM21216_SETUP.EXE, qui installe les drivers VCP (Virtual COM Port) du module FTDI. Cela a permis au système d'exploitation de reconnaître le convertisseur USB-série comme un port COM virtuel et d'établir une communication série stable avec la carte FOX G20.

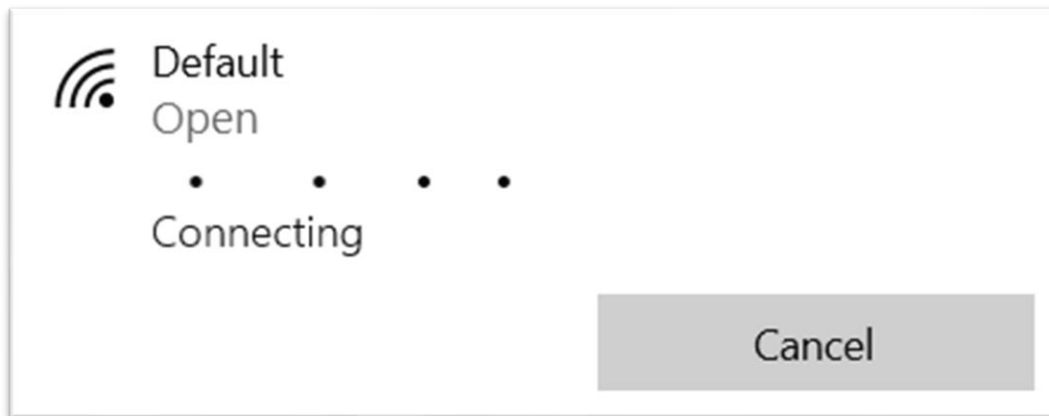


Ensuite, nous avons installé PuTTY, qui est un client de terminal série (Serial Terminal) gratuit, léger et extrêmement fiable. Ce logiciel permet au système d'exploitation Windows d'établir une communication bidirectionnelle stable et en temps réel avec la console série de la carte FOX G20 à travers le port COM virtuel créé par le pilote FTDI.

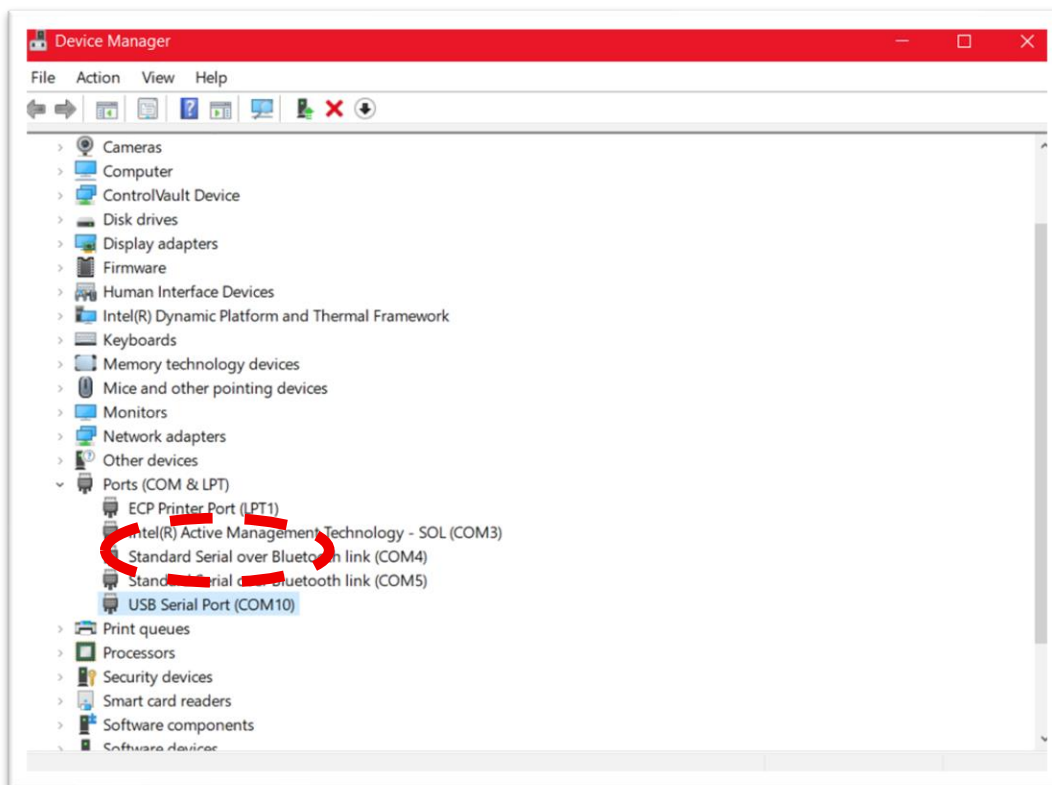


Connexion au Wifi :

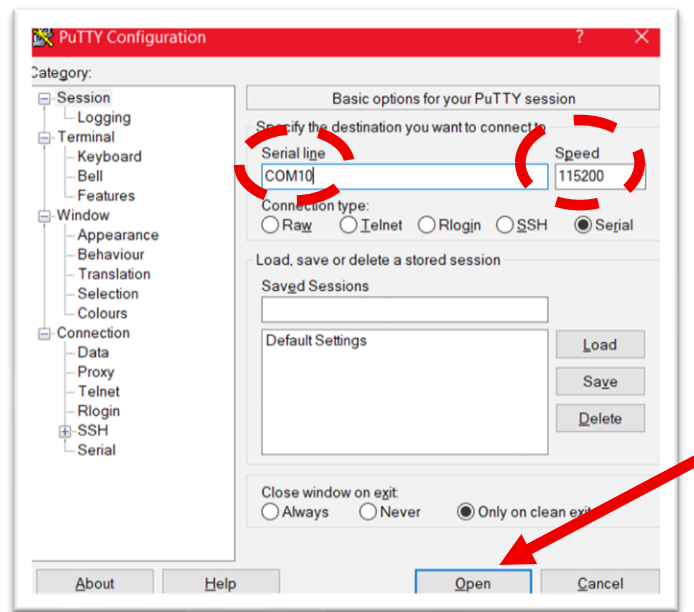
Ensuite, après avoir installé avec succès le pilote CDM21216_SETUP.EXE et le client PuTTY, nous nous sommes connectés au routeur Wi-Fi nommé « Default », afin d'intégrer le PC hôte dans le même réseau local que la carte FOX G20.



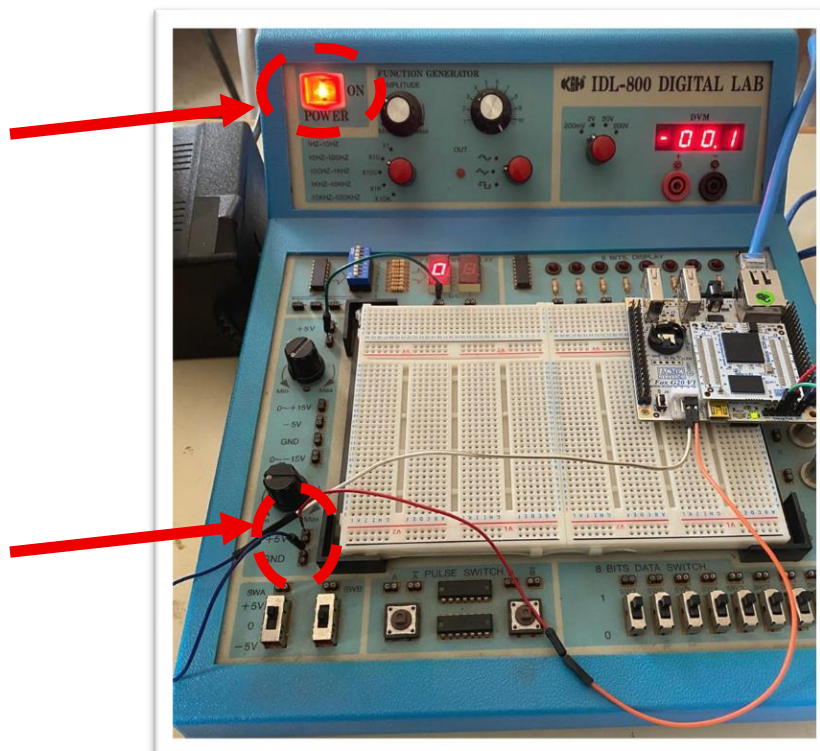
Ensuite, nous avons extrait le numéro du port du module **FTDI** connecté à partir du **Gestionnaire de périphériques** (COM10 dans notre cas).



Nous avons alors lancé le logiciel PuTTY sur le PC hôte, sélectionné le port COM10 et configuré la vitesse de communication à 115200 bauds.



Ensuite, après nous avons cliquer sur Open, nous avons mis sous tension la carte FOX G20 à l'aide de IDL-800 DIGITAL LAB. Mais avant, Nous avons bien confirmé que la carte microSD contenant l'OS Linux était insérée dans le lecteur situé à l'arrière de la carte.



On remarque l'allumage du voyant vert, qui indique que la carte est correctement alimentée. Le voyant rouge s'est également allumé en clignotant, confirmant que le système Linux embarqué démarre avec succès et que la carte est pleinement opérationnelle. Par la suite, la fenêtre PuTTY commence à capturer l'ensemble des données de débogage, comme le montre la capture d'écran ci-après.

```
COM10 - PuTTY
atmel_mci fffa8000.mmc: no DMA channel available
atmel_mci fffa8000.mmc: using PDC
atmel_mci fffa8000.mmc: Atmel MCI controller at 0xfffa8000 irq 32, 1 slots
Waiting for root device /dev/mmcblk0p2...
mmc0: host does not support reading read-only switch. assuming write-enable.
mmc0: new SD card at address a7c4
mmcblk0: mmc0:a7c4 S01G 942 MiB
mmcblk0: p1 p2
EXT4-fs (mmcblk0p2): mounting ext3 file system using the ext4 subsystem
EXT4-fs (mmcblk0p2): mounted filesystem with ordered data mode. Opts: (null)
VFS: Mounted root (ext3 filesystem) on device 179:2.
devtmpfs: mounted
Freeing unused kernel memory: 152K (c045f000 - c0485000)
INIT: version 2.88 booting
[info] Using makefile-style concurrent boot in runlevel S.
[....] Starting the hotplug events dispatcher: udevdudevd[519]: starting version
175
. ok
[....] Synthesizing the initial hotplug events...ohci_hcd: USB 1.1 'Open' Host C
ontroller (OHCI) Driver
at91_ohci 500000.ohci: AT91 OHCI
at91_ohci 500000.ohci: new USB bus registered, assigned bus number 1
at91_ohci 500000.ohci: irq 34, io mem 0x00500000
usb usb1: New USB device found, idVendor=1d6b, idProduct=0001
usb usb1: New USB device strings: Mfr=3, Product=2, SerialNumber=1
usb usb1: Product: AT91 OHCI
usb usb1: Manufacturer: Linux 3.11.6 ohci_hcd
usb usb1: SerialNumber: at91
hub 1-0:1.0: USB hub found
hub 1-0:1.0: 2 ports detected
udc: at91_udc version 3 May 2006
done.
[ ok ] Waiting for /dev to be fully populated...done.
[ ok ] Activating swap...done.
EXT4-fs (mmcblk0p2): re-mounted. Opts: (null)
[....] Checking root file system...fsck from util-linux 2.20.1
rootfs: clean, 13915/54544 files, 109651/217856 blocks
done.
EXT4-fs (mmcblk0p2): re-mounted. Opts: (null)
[ ok ] Cleaning up temporary files... /tmp.
[ ok ] Activating lvm and md swap...done.
[....] Checking file systems...fsck from util-linux 2.20.1
fsck.ext4: No such file or directory while trying to open /dev/mmcblk0p3
Possibly non-existent device?
fsck died with exit status 8
failed (code 8).
[FAIL] File system check failed. A log is being saved in /var/log/fsck/checkfs if that location is writable. Please repair the file system manually. ... failed!
[warn] A maintenance shell will now be started. CONTROL-D will terminate this shell and resume system boot. ... (warning).
Give root password for maintenance
(or type Control-D to continue):
```

Ici, nous remarquons que le système a rencontré une erreur de démarrage Linux : le système de fichiers racine (/dev/mmcblk0p3) n'a pas pu être monté en lecture-écriture. Comme ce n'est pas une erreur critique, nous pouvons simplement continuer le démarrage et ignorer la réparation manuelle en appuyant sur Ctrl+D.

```
Give root password for maintenance
(or type Control-D to continue):
[....] Mounting local filesystems...FAT-fs (mmcblk0p1): Volume was not properly unmounted. Some data may be corrupt. Please run fsck.
mount: special device /dev/mmcblk0p3 does not exist
failed.
[ ok ] Activating swapfile swap...done.
[ ok ] Cleaning up temporary files...
[ ok ] Setting kernel variables ...done.
[....] Configuring network interfaces...IPv6: ADDRCONF(NETDEV_UP): eth0: link is not ready
done.
[ ok ] Cleaning up temporary files...
INIT: Entering runlevel: 2
[info] Using makefile-style concurrent boot in runlevel 2.
mactb fffc4000.ethernet eth0: link up (10/Half)
IPv6: ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready
IPv6: eth0: IPv6 duplicate address fe80::204:25ff:feaa:317 detected!
[ ok ] Starting system message bus: dbus.
[ ok ] Starting web server: lighttpd.
[ ok ] Starting Avahi mDNS/DNS-SD Daemon: avahi-daemon.
[ ok ] Starting OpenBSD Secure Shell server: sshd.
Debian GNU/Linux (Acme Systems AriaG25-1403) netusg20 ttyS0
netusg20 login:
```

Ici, nous voyons que Le démarrage s'est poursuivi normalement : swap activé, services lancés (lighttpd, avahi...), et nous arrivons enfin sur l'invite de connexion Debian. Les identifiants utilisés étaient :

Login : root

Password : netusg20

Ici, nous pouvons voir que la connexion a réussi :

```
netusg20 login: root
Password:
Last login: Thu Jan  1 01:02:38 CET 1970 on ttyS0
Linux netusg20 3.11.6 #2 Fri Aug 22 15:01:19 CEST 2014 armv5tej1

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@netusg20:~#
```

Nous avons exécuté la commande « **ifconfig eth0** » sans forcer une adresse IP spécifique, mais elle nous a attribué la même adresse qu'une autre carte, ce qui a causé un conflit, nous avons donc dû la modifier.

```
root@netusg20:~# ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 00:04:25:aa:03:17
          inet addr:192.168.2.100  Bcast:192.168.2.255  Mask:255.255.255.0
          inet6 addr: fe80::204:25ff:feaa:317/64  Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:224 errors:34 dropped:0 overruns:0 frame:0
          TX packets:34 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:31708 (30.9 KiB)  TX bytes:6625 (6.4 KiB)
          Interrupt:30 Base address:0x4000
```

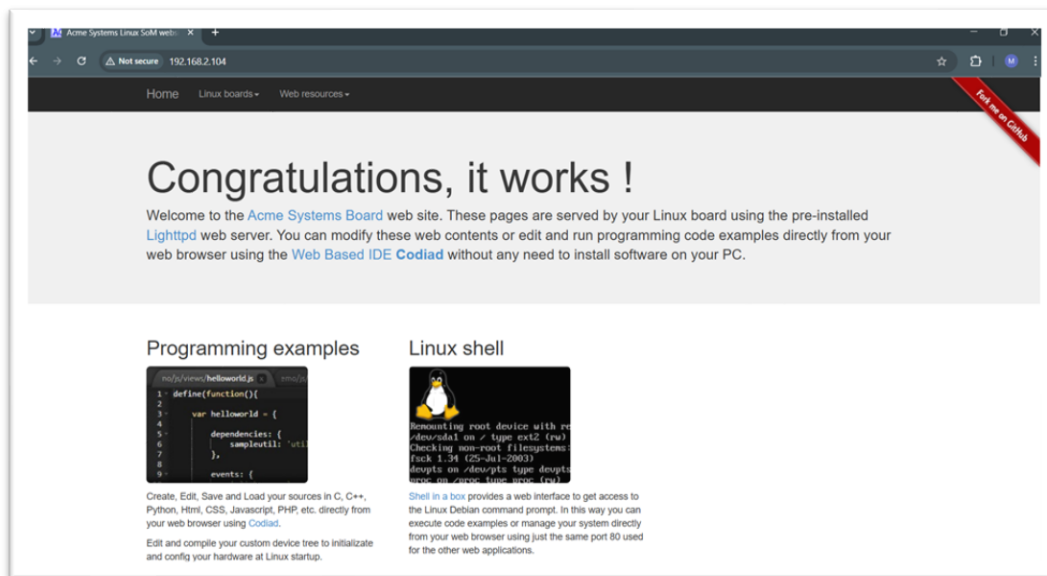
Pour résoudre ce problème, nous avons simplement répété la même commande et ajouté simplement l'adresse IP souhaitée après celle-ci (**192.168.2.104**) :

```
root@netusg20:~# ifconfig eth0 192.168.2.104
```

Ici, nous avons exécuté à nouveau la commande **ifconfig eth0** afin de vérifier l'adoption de notre nouvelle adresse IP. Nous pouvons constater que celle-ci a été appliquée avec succès.

```
root@netusg20:~# ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 00:04:25:aa:03:17
          inet addr:192.168.2.104  Bcast:192.168.2.255  Mask:255.255.255.0
          inet6 addr: fe80::204:25ff:feaa:317/64  Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1117 errors:128 dropped:0 overruns:0 frame:0
          TX packets:128 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:183871 (179.5 KiB)  TX bytes:20275 (19.7 KiB)
          Interrupt:30 Base address:0x4000
```

Après avoir ouvert Google Chrome et saisi notre adresse IP unique, nous avons été informés que nous étions connectés avec succès à l'IDE Web **Codiad** :

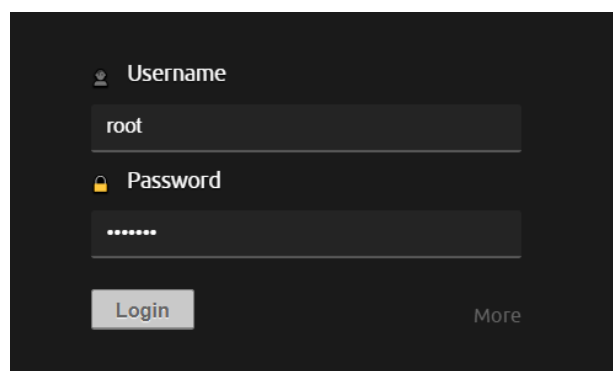


Ici, Nous avons extrait les identifiants de connexion suivants :

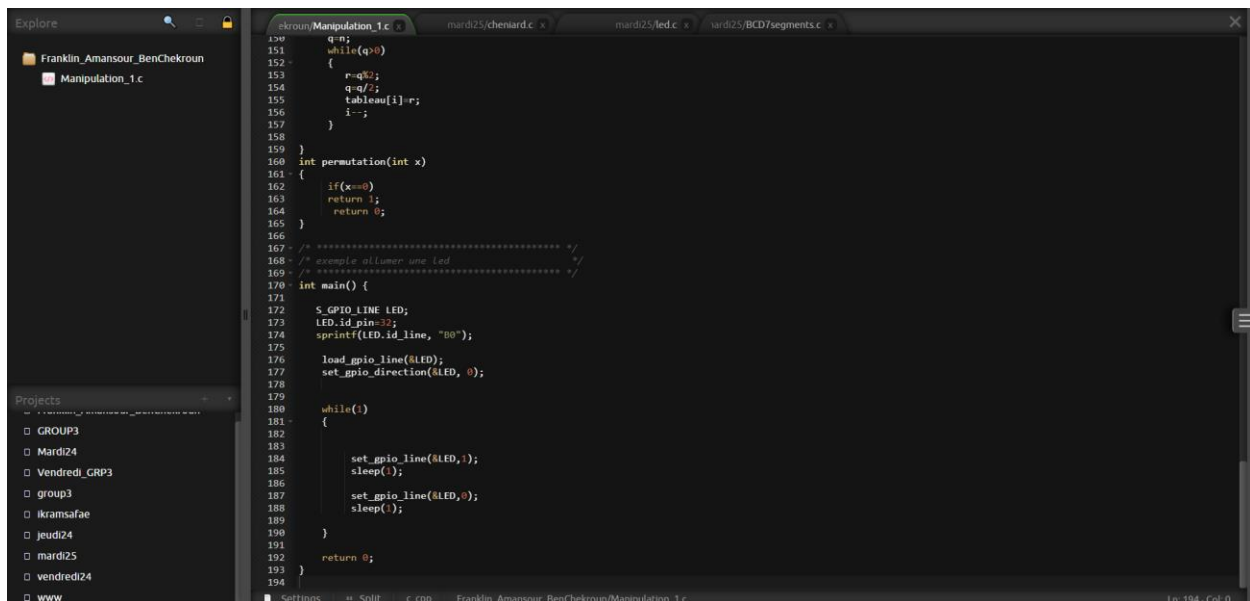
Nom d'utilisateur : root

Mot de passe : ariag25

Après nous avoir cliqué sur Codiad Web IDE, nous avons été redirigés vers la page où nous avons entré les identifiants de connexion :



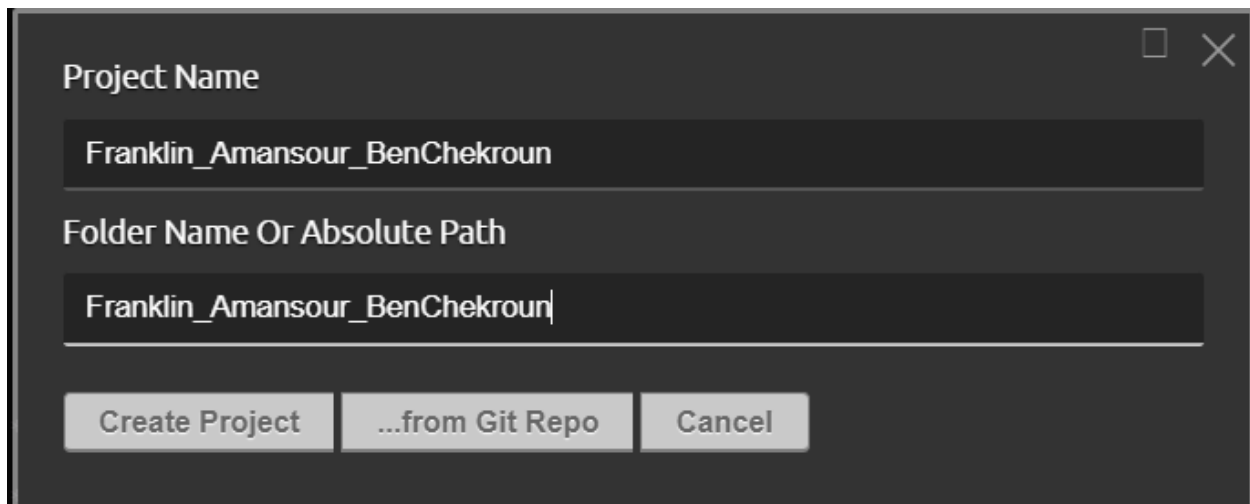
Après s'être connecté, nous trouvons une interface comme celle-ci :



The screenshot shows an IDE with a dark theme. On the left, there's a sidebar with 'Explore' and 'Projects' views. The 'Projects' view shows a tree structure with folders like 'GROUP3', 'Mardi24', 'Vendredi_GRP3', 'group3', 'IkramSafae', 'jeudi24', 'mardi25', 'vendredi24', and 'www'. The main editor area displays a C program named 'Manipulation_1.c'. The code includes a permutation function, a GPIO setup for an LED, and a main loop that toggles the LED state. The status bar at the bottom indicates 'Settings', 'Split', 'C.cpp', and the file path 'Franklin_Amansour_BenChekroun/Manipulation_1.c'.

```
150  q=q;
151  while(q!=0)
152  {
153      r=q/2;
154      q=q/2;
155      tableau[i]=r;
156      i--;
157  }
158  }
159  }
160  int permutation(int x)
161  {
162      if(x==0)
163          return 1;
164      return 0;
165  }
166  /*
167  // exemple allumer une led
168  //
169  */
170  int main() {
171      S_GPIO_LINE LED;
172      LED.id_pin=32;
173      sprintf(LED.id_line, "%d", LED.id_pin);
174      load_gpio_line(LED);
175      set_gpio_direction(LED, 0);
176      while(1)
177      {
178          set_gpio_line(LED, 1);
179          sleep(1);
180          set_gpio_line(LED, 0);
181          sleep(1);
182      }
183      return 0;
184  }
```

Nous avons commencé par créer un nouveau dossier, nommé « Franklin_Amansour_BenChekroun » :



The screenshot shows a 'Create Project' dialog box with a dark theme. It has two input fields: 'Project Name' and 'Folder Name Or Absolute Path'. Both fields contain the text 'Franklin_Amansour_BenChekroun'. At the bottom, there are three buttons: 'Create Project', '...from Git Repo', and 'Cancel'.

Project Name

Franklin_Amansour_BenChekroun

Folder Name Or Absolute Path

Franklin_Amansour_BenChekroun

Create Project ...from Git Repo Cancel

Manipulation 1 : LED Clignotante

But de la manipulation :

Dans cette manipulation, nous avons travaillé avec un système embarqué pour contrôler une ligne GPIO (General Purpose Input/Output). L'objectif était d'interagir directement avec le matériel en initialisant une broche GPIO, en configurant sa direction et en contrôlant son état numérique. Pour la démonstration, nous avons utilisé la ligne GPIO pour contrôler une LED, en la faisant s'allumer et s'éteindre périodiquement. Cette manipulation nous a permis d'observer comment les composants matériels peuvent être gérés via les lignes GPIO et d'acquérir une expérience pratique du contrôle des systèmes embarqués et de l'interface matérielle.

Logiciels embarqués :

Aperçu

Ce logiciel embarqué est un petit programme C conçu pour contrôler les broches GPIO via l'interface linux sysfs. Il exporte une broche GPIO, configure sa direction (entrée/sortie), écrit les valeurs de sortie et lit les états des broches. Le programme fait la démonstration du contrôle numérique de base, comme l'activation d'une LED, et inclut des fonctions d'assistance pour la manipulation GPIO et les opérations simples sur les bits.

Fonctions utilisées

➤ **load_gpio_line()**

Cette fonction est utilisée pour « activer » une broche GPIO afin que le système puisse l'utiliser. Il écrit le numéro de broche dans le fichier `/sys/class/gpio/export`, qui indique au noyau Linux de créer le répertoire pour cette broche. Après cela, toutes les autres opérations (direction, valeur) deviennent possibles.

➤ **set_gpio_direction()**

Cette fonction définit si la broche GPIO doit être utilisée comme entrée ou comme sortie. Il construit le chemin d'accès au fichier de direction à l'aide de l'ID de la broche, puis y écrit « in » ou « out ». Cette étape définit le comportement électrique de la broche et est nécessaire avant d'écrire des valeurs.

➤ **set_gpio_line()**

Cette fonction est responsable de l'écriture d'une valeur sur la broche GPIO, soit 0 (LOW) ou 1 (HIGH). Il ouvre le fichier de valeurs associé à la broche et écrit le numéro choisi. Il stocke également la valeur à l'intérieur de la structure afin que le programme se souvienne du dernier état de la broche.

➤ **get_gpio_line()**

Cette fonction lit la valeur réelle présente sur une broche GPIO. Il ouvre le fichier de valeurs de la broche, lit un seul caractère ('0' ou '1'), le convertit en entier équivalent et le stocke dans la structure. Il est utilisé pour vérifier l'état d'une broche d'entrée.

Fonction principale main()

La fonction main() initialise et contrôle une LED à l'aide d'une broche GPIO. Elle suit les étapes suivantes :

1. Initialisation de la broche GPIO

- Création d'une structure S_GPIO_LINE nommée LED.
- Attribution de id_pin = 32 et id_line = "B0".

2. Activation et configuration de la broche

- Appel de load_gpio_line(&LED) pour exporter la broche GPIO et la rendre accessible via sysfs.
- Appel de set_gpio_direction(&LED, OUT) pour configurer la broche comme sortie.

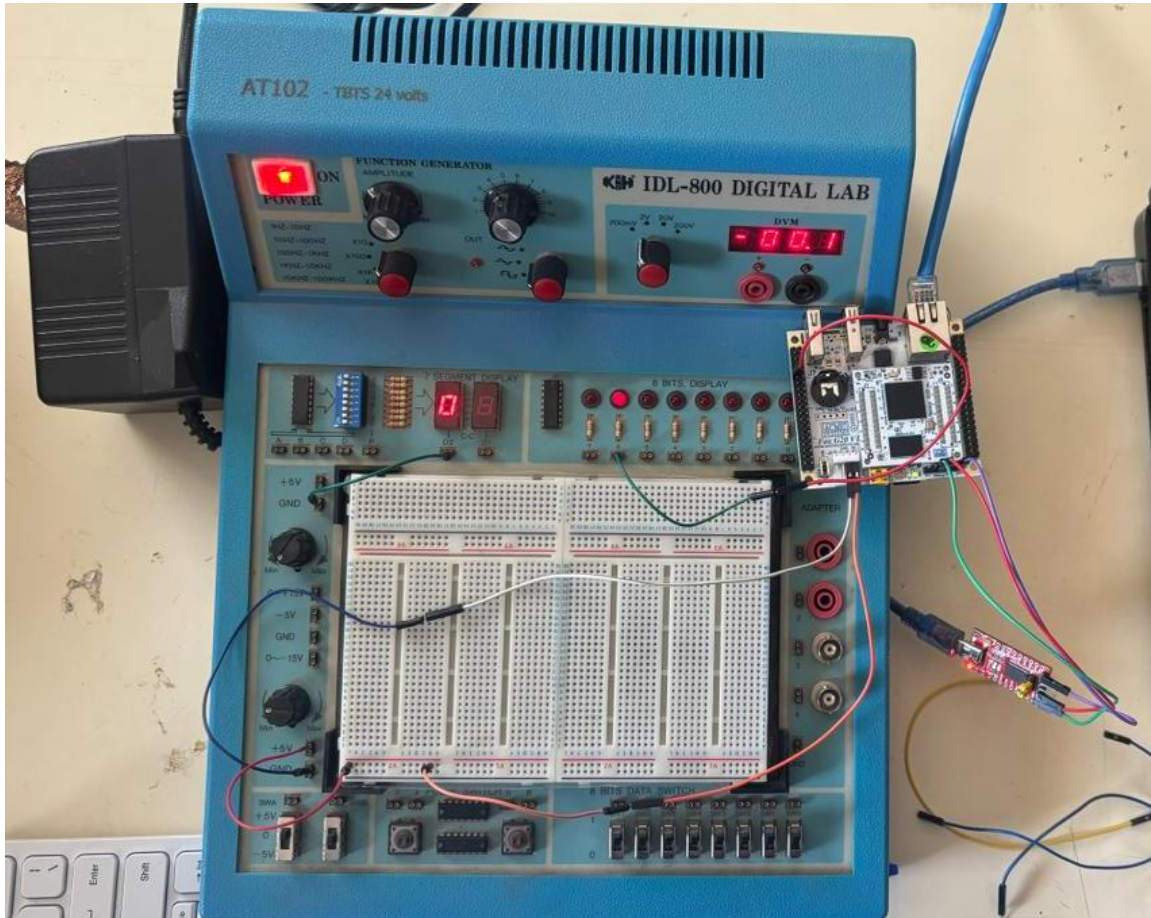
3. Boucle principale de contrôle

Entrée dans une boucle infinie pour faire clignoter la LED :

- set_gpio_line(&LED, 1) → met la broche à HIGH, LED allumée.
- sleep(1) → temporisation de 1 seconde.
- set_gpio_line(&LED, 0) → met la broche à LOW, LED éteinte.
- sleep(1) → temporisation de 1 seconde.

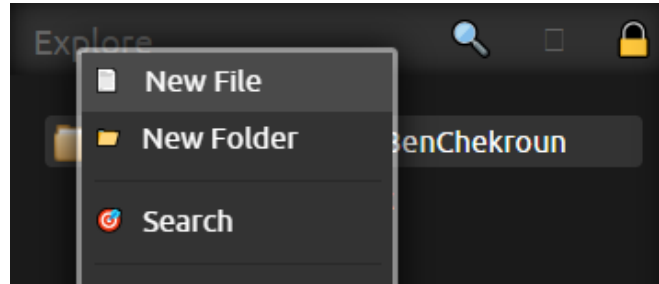
Réalisation physique

Voici comment nous avons réalisé physiquement la manipulation 1 en utilisant le IDL-800 Digital Lab et la carte Fox G20.

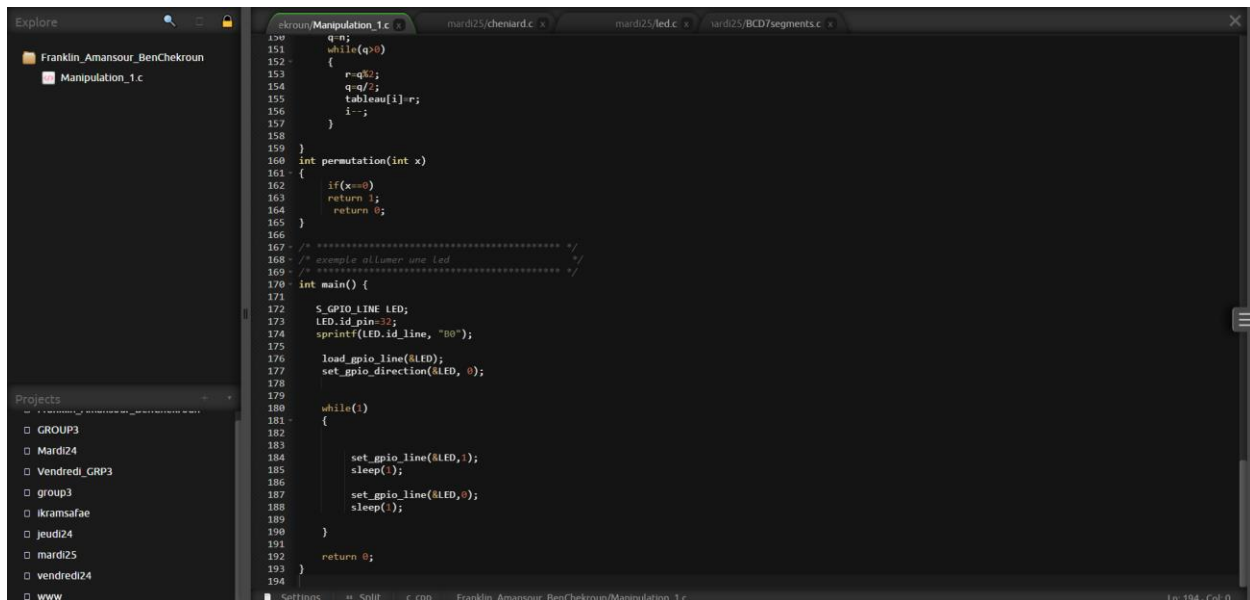


Configuration

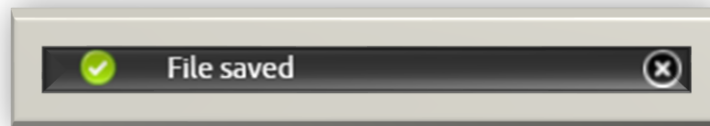
Nous avons fait un clic droit sur le nom du projet pour créer un nouveau fichier C nommé M1.c :



Pour la manipulation 1 (LED clignotante), nous avons collé le code de driver.txt dans M1.c



Puis nous enregistrons le fichier en cliquant sur CTRL + S, puis nous voyons cette confirmation apparaître :



Après, nous avons procédé au processus de compilation en entrant la commande « **gcc -o exe M1.c** »

Enfin, le programme compilé a été exécuté sur le FOX en exécutant la commande « **./exe** »

```
root@netusg20:/var/Codiad/workspace/Franklin_Amansour_BenChekroun#  
.cot@netusg20:/var/Codiad/workspace/Franklin_Amansour_BenChekroun# gcc -o exe M1  
root@netusg20:/var/Codiad/workspace/Franklin_Amansour_BenChekroun# ./exe  
path:/sys/class/gpio/gpioB0/direction
```

Remarque : à cause d'un bug dans *PuTTY*, nous avons remarqué qu'une partie de nos commandes n'était pas correctement écrite dans la console mais elles étaient quand même capturées correctement et sommes donc passées à l'étape suivante.

Conclusion Partielle

En conclusion, cette manipulation nous a permis de maîtriser le contrôle d'une ligne GPIO sur un système embarqué. Nous avons appris à initialiser une broche, à définir sa direction et à gérer son état numérique pour piloter un composant matériel, ici une LED. Cette expérience pratique nous a donné une meilleure compréhension de l'interaction directe avec le matériel et constitue une base solide pour des applications plus complexes sur les systèmes embarqués.

Manipulation 2 : Chenillard Lumineux

But de la manipulation :

Dans cette manipulation, nous avons capitalisé sur les compétences acquises lors de la première manipulation en passant du contrôle d'une seule LED à la gestion simultanée de trois LED, chacune connectée à une broche GPIO distincte d'un système embarqué. L'objectif était de réaliser un effet de chenillard lumineux, où les trois LED s'allument successivement l'une après l'autre, de gauche à droite), puis s'éteignent, créant une animation fluide et continue.

Logiciels embarqués :

Aperçu

Puisque toutes les fonctions nécessaires sont déjà implémentées dans le code de la manipulation 1, nous réutilisons intégralement ce socle, aucune nouvelle fonction n'est ajoutée, et seule la fonction `main()` est modifiée pour produire l'effet de chenillard.

Pour ce faire, on a :

1. Nommé des entrées/sorties et des variables (I/O)
2. Utilisé le décalage de bit
3. Fait une fonction

Fonction principale `main()`

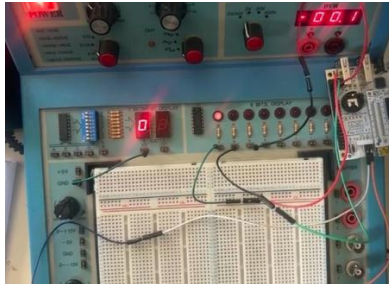
Nous avons adapté la logique pour gérer trois LEDs allumées successivement, effet de chenillard.

Pour cela, nous avons :

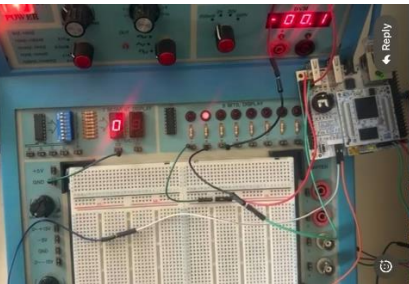
- Déclaré trois structures `S_GPIO_LINE` (LED1, LED2, LED3) avec leurs numéros de broches respectifs (50, 51, 48) et identifiants ("B18", "B19", "B16"),
- Exporté et configuré chaque broche comme sortie via `load_gpio_line()` et `set_gpio_direction(..., 0)`,
- Modifié la boucle infinie pour allumer chaque LED l'une après l'autre :
 - Allumage de LED1 pendant 1 seconde, puis extinction immédiate,
 - Allumage de LED2 pendant 1 seconde, puis extinction,
 - Allumage de LED3 pendant 1 seconde, puis extinction,
 - Répétition du cycle.

Réalisation physique

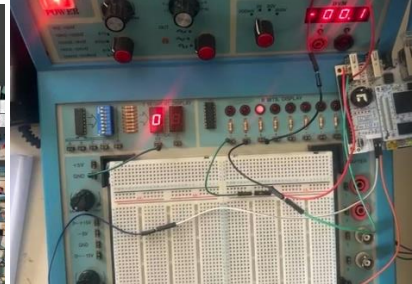
Voici comment nous avons réalisé physiquement la manipulation 2 en utilisant le IDL-800 Digital Lab et la carte Fox G20.



(1)



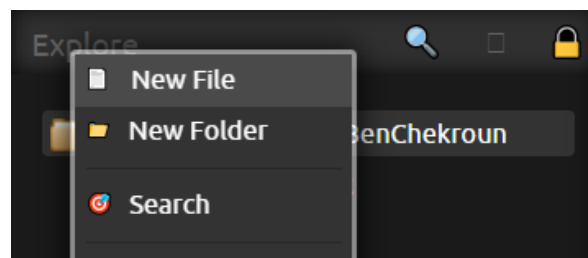
(2)



(3)

Configuration

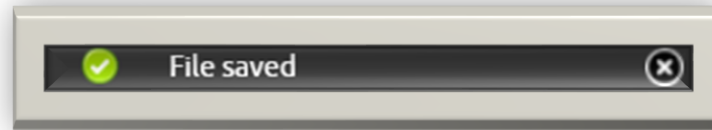
Nous avons fait un clic droit sur le nom du projet pour créer un nouveau fichier C nommé M2.c :



Pour la manipulation 2 (Chenillard Lumineux), nous avons collé le code dans M2.c

```
170 int main() {
171
172     S_GPIO_LINE LED1, LED2, LED3;
173     LED1.id_pin=50;
174     LED2.id_pin=51;
175     LED3.id_pin=48;
176
177
178     sprintf(LED1.id_line, "B10");
179     sprintf(LED2.id_line, "B19");
180     sprintf(LED3.id_line, "B16");
181
182     load_gpio_line(&LED1);
183     set_gpio_direction(&LED1, 0);
184
185     load_gpio_line(&LED2);
186     set_gpio_direction(&LED2, 0);
187
188     load_gpio_line(&LED3);
189     set_gpio_direction(&LED3, 0);
190
191
192     while(1)
193     {
194
195
196         set_gpio_line(&LED1,1);
197         sleep(1);
198
199         set_gpio_line(&LED1,0);
200         sleep(0);
201
202         set_gpio_line(&LED2,1);
203         sleep(1);
204
205         set_gpio_line(&LED2,0);
206         sleep(0);
207
208         set_gpio_line(&LED3,1);
209         sleep(1);
210
211         set_gpio_line(&LED3,0);
212         sleep(0);
213
214     }
215
216     return 0;
217 }
218
```

Puis nous enregistrons le fichier en cliquant sur CTRL + S, puis nous voyons cette confirmation apparaître :



Après, nous avons procédé au processus de compilation en entrant la commande « **gcc -o exe2 M2.c** »

Enfin, le programme compilé a été exécuté sur le FOX en exécutant la commande « **./exe2** »

```
2.ct@netusg20:/var/Codiad/workspace/Franklin_Amansour_BenChekroun# gcc -o exe2 M
root@netusg20:/var/Codiad/workspace/Franklin_Amansour_BenChekroun# ./exe2
path:/sys/class/gpio/pioB18/direction
path:/sys/class/gpio/pioB19/direction
path:/sys/class/gpio/pioB16/direction
```

Conclusion Partielle

Au terme de cette manipulation, nous avons réussi à transformer un simple clignotement d'une LED en un effet de chenillard séquentiel sur trois LEDs. En réutilisant le socle fonctionnel de la manipulation 1, nous avons adapté la fonction main() pour implémenter une séquence lumineuse fluide (LED1 → LED2 → LED3) avec extinction immédiate avant allumage de la suivante.

Manipulation 3 : Compteur

But de la manipulation :

À partir de la base du chenillard à trois LEDs (manipulation 2), nous avons réalisé un compteur de 0 à 3 en commandant un afficheur 7 segments via des broches GPIO, en générant les motifs binaires correspondant à chaque chiffre et en les affichant séquentiellement avec une temporisation.

Logiciels embarqués :

Aperçu

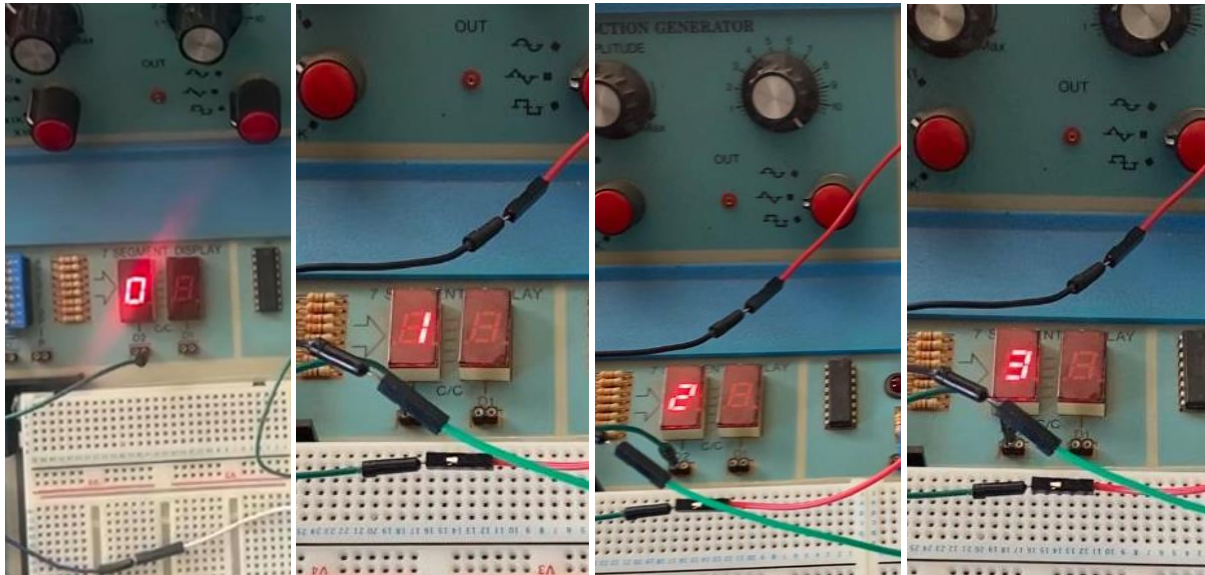
Lors de la manipulation 3, nous nous sommes rendu compte qu'en connectant les lignes GPIO à un décodeur BCD à 7 segments, chaque combinaison d'états d'entrée pouvait représenter directement un nombre sur l'écran. Nous avons mis à jour la fonction principale afin qu'elle définisse les lignes GPIO en fonction des valeurs BCD de 0 à 3, et que le décodeur allume automatiquement les segments corrects. Cela nous a fait comprendre que nous n'avions plus besoin de contrôler chaque segment individuellement, il suffisait de régler les entrées BCD pour afficher les chiffres, ce qui simplifiait le programme et s'appuyait naturellement sur ce que nous avons fait avec les LED lors de la manipulation précédente.

Fonction principale main()

Dans cette manipulation, nous avons réutilisé la même structure de boucle que la commande LED précédente, mais l'avons adaptée pour l'affichage à 7 segments. La boucle `while(1)` fait défiler continuellement les nombres en définissant les lignes GPIO correspondantes HIGH ou LOW. Au lieu d'allumer des LED individuelles de manière séquentielle, plusieurs GPIO sont désormais réglés simultanément en fonction de l'entrée BCD pour chaque numéro, et les délais `sleep()` garantissent que les chiffres sont visibles.

Réalisation physique

Voici comment nous avons réalisé physiquement la manipulation 2 en utilisant le IDL-800 Digital Lab et la carte Fox G20.



(1)

(2)

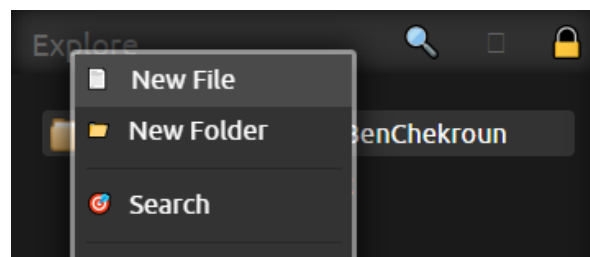
(3)

(4)

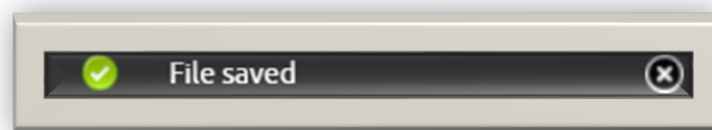
...

Configuration

Nous avons fait un clic droit sur le nom du projet pour créer un nouveau fichier C nommé M3.c :



Pour la manipulation 3, nous avons collé le code dans M3.c, puis nous enregistrons le fichier en cliquant sur CTRL + S, puis nous voyons cette confirmation apparaître :



Après, nous avons procédé au processus de compilation en entrant la commande
« **gcc -o exe2 M3.c** »

Enfin, le programme compilé a été exécuté sur le FOX en exécutant la commande « **./exe2** »

```
2.ct@netusg20:/var/Codiad/workspace/Franklin_Amansour_BenChekroun# gcc -o exe2 M
root@netusg20:/var/Codiad/workspace/Franklin_Amansour_BenChekroun# ./exe2
path:/sys/class/gpio/pioB18/direction
path:/sys/class/gpio/pioB19/direction
path:/sys/class/gpio/pioB16/direction
```

Conclusion Partielle

Nous avons réussi à transformer le chenillard à 3 LEDs en un compteur contrôlé en BCD affichant 0 à 3 sur un afficheur 7 segments.

Conclusion Générale

Tout au long de ce TP, nous avons utilisé la carte FOX G20 pour progresser du contrôle élémentaire d'une LED (clignotement) au chenillard séquentiel à 3 LEDs (allumage l'un après l'autre), puis réalisé un compteur BCD de 0 à 3 sur afficheur 7 segments, en réutilisant systématiquement les fonctions GPIO via sysfs. Nous avons maîtrisé l'export, la configuration et la commande des broches, la logique séquentielle, les motifs binaires et le codage BCD, tout en construisant un code clair, modulaire et évolutif. Ce parcours pratique sur un véritable système embarqué a renforcé nos compétences en programmation système en C sous Linux, et transformé des concepts théoriques en applications visuelles concrètes et fonctionnelles.